Dynamic Application Security Testing Report

acme/app

FAILED	6 Alerts	1 High	0 Medium	3 Low	2 Informational

Target

Scanned URL	http://127.0.0.1:60369
Server	PHP's built-in web server (APP_ENV=test)
Scan	OWASP ZAP: check the requests and responses of the End-to-End tests, and attack them (up to 5 min)
End-to-End tests	vendor/bin/phpunit --testsuite EndToEnd
Login	None (as a visitor)
URLs with alerts	6
Fails on	High or higher

Run

Date	2026-10-17 15:04:13 – 15:10:13 (UTC)
Duration	5m 59s
Environment	Local (macOS 27.0 arm64)
Software	PHP 8.5.11 OWASP ZAP 2.17.0 pieceofcake2/security-dast 1.0.0+no-version-set

The full scan attacks the pages and their parameters, e.g. with scripts and SQL, as an attacker would; it finds many vulnerabilities, but not all, e.g. those of the pages that it did not crawl.

Alerts by risk

1. High

No.	Alert	Risk	Requests
1-1	Cross Site Scripting (Reflected)	High	1

2. Low

No.	Alert	Risk	Requests
2-1	Cookie without SameSite Attribute	Low	1
2-2	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	2
2-3	X-Content-Type-Options Header Missing	Low	5

3. Informational

No.	Alert	Risk	Requests
3-1	Session Management Response Identified	Informational	1
3-2	User Controllable HTML Element Attribute (Potential XSS)	Informational	3

1. High

1-1 Cross Site Scripting (Reflected)

High

Alert ID	40012
Risk	High
Confidence	Medium
CWE	CWE-79
WASC	WASC-8
Requests	1

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.
When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.

There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.
Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.
Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.

Solution

Phase: Architecture and Design
Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.
Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket.

Phases: Implementation; Architecture and Design
Understand the context in which your data will be used and the encoding that will be expected. This is especially

important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies.
For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters.
Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed.

Phase: Architecture and Design
For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.

If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Phase: Implementation
For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping.

To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHTTPRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set.

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Requests

POST	http://127.0.0.1:60369/**contacts/add**
Parameter	data[Contact][message]
Attack	<scRipt>alert(1);</scRipt>
Evidence	<scRipt>alert(1);</scRipt>

References

- https://www.zaproxy.org/docs/alerts/40012/
- https://owasp.org/www-community/attacks/xss/
- https://cwe.mitre.org/data/definitions/79.html

2. Low

2-1 Cookie without SameSite Attribute

Low

Alert ID	10054-1
Risk	Low
Confidence	Medium
CWE	CWE-1275
WASC	WASC-13
Requests	1

A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

Solution

Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

Requests

GET http://127.0.0.1:60369/contacts/add	
Parameter	CAKEPHP
Evidence	Set-Cookie: CAKEPHP

References

- https://www.zaproxy.org/docs/alerts/10054/
- https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site

2. Low

2-2 Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)

Low

Alert ID	10037
Risk	Low
Confidence	Medium
CWE	CWE-497
WASC	WASC-13
Requests	2

The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.

Solution

Ensure that your web server, application server, load balancer, etc. is configured to suppress "X-Powered-By" headers.

Requests

GET	http://127.0.0.1:60369/contacts/add
Evidence	X-Powered-By: PHP/8.5.11

POST	http://127.0.0.1:60369/contacts/add
Evidence	X-Powered-By: PHP/8.5.11

References

- https://www.zaproxy.org/docs/alerts/10037/
- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework
- https://www.troyhunt.com/shhh-dont-let-your-response-headers/

2. Low

2-3 X-Content-Type-Options Header Missing

Low

Alert ID	10021
Risk	Low
Confidence	Medium
CWE	CWE-693
WASC	WASC-15
Requests	5

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Solution

Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.
If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Requests

GET http://127.0.0.1:60369/css/cake.css	
Parameter	x-content-type-options
Other info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

GET http://127.0.0.1:60369/css/fonts.css	
Parameter	x-content-type-options
Other info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

GET http://127.0.0.1:60369/css/milligram.min.css	
Parameter	x-content-type-options
Other info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

GET	http://127.0.0.1:60369/**css/normalize.min.css**
Parameter	x-content-type-options
Other info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

GET	http://127.0.0.1:60369/**font/raleway-latin.woff2**
Parameter	x-content-type-options
Other info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

References

- https://www.zaproxy.org/docs/alerts/10021/
- https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85)
- https://owasp.org/www-community/Security_Headers

3. Informational

3-1 Session Management Response Identified

Informational

Alert ID	10112
Risk	Informational
Confidence	Medium
Requests	1

The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.

Solution

This is an informational alert rather than a vulnerability and so there is nothing to fix.

Requests

GET http://127.0.0.1:60369/contacts/add	
Parameter	CAKEPHP
Evidence	CAKEPHP
Other info	cookie:CAKEPHP

References

- https://www.zaproxy.org/docs/alerts/10112/
- https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/

3. Informational

3-2 User Controllable HTML Element Attribute (Potential XSS)

Informational

Alert ID	10031
Risk	Informational
Confidence	Low
CWE	CWE-20
WASC	WASC-20
Requests	3

This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

Solution

Validate all input and sanitize output it before writing to any HTML attributes.

Requests

POST http://127.0.0.1:60369/contacts/add	
Parameter	_method
Other info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://host.docker.internal:60369/contacts/add appears to include user input in: a(n) [form] tag [method] attribute The user input found was: _method=POST The user-controlled value was: post

POST	http://127.0.0.1:60369/contacts/add
Parameter	data[_Token][fields]
Other info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://host.docker.internal:60369/contacts/add appears to include user input in: a(n) [input] tag [value] attribute The user input found was: data[_Token][fields]=7b6fbe16e9adac1e320c118cd2348cb3362680ec%3A The user-controlled value was: 7b6fbe16e9adac1e320c118cd2348cb3362680ec%3a

POST	http://127.0.0.1:60369/contacts/add
Parameter	data[Contact][message]
Other info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://host.docker.internal:60369/contacts/add appears to include user input in: a(n) [input] tag [value] attribute The user input found was: data[Contact][message]=Hello The user-controlled value was: hello

References

- https://www.zaproxy.org/docs/alerts/10031/
- https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html